Assignment 4 - Histogram

> DUE Wednesday, November 18 at 11:59:59 PM Pacific Time

Overview

The objective of this assignment is to implement histogram kernels and demonstrate the usage of atomic operations and privatization. You will implement two versions, a basic histogram and a privatized histogram, and compare their performance on a real GPU and in GPGPU-Sim.

Basic Histogram

In the first step, you will implement a basic histogram kernel. Each thread reads its input elements and adds each one directly to the histogram in global memory with `atomicAdd`. Threads that hit the same bin at the same time contend for the same global memory location.

Privatized Histogram

In the second step, you will implement a histogram with the **privatization** pattern. Each block:

- zeroes its own private histogram in shared memory,

- counts its input elements into the private histogram with `atomicAdd`,
- adds its private histogram to the global histogram `bins` with `atomicAdd`.

Remember `__syncthreads()` between these phases. The number of bins can be larger or smaller than the 512 threads of a block, so make sure your loops over the bins cover every bin.

Github Classroom

1. For this lab, we will be using Classroom 50 (replacement for GitHub Classroom).

   > Please accept the assignment by clicking the following link: https://classroom50.org/UCR-CSEE217/cs-ee217-fall-2026/assignments/histogram/accept

   Once you accept the assignment, a private GitHub repository will automatically be created with the starter code. Simply `git clone` it to copy the starter code to Bender. It contains:

   - `kernel.cu`: where you write your code
   - `main.cu`, `support.cu`, `support.h`, `Makefile`: provided; do not modify
   - `REPORT.md`, `plots/`: your written report (see Report)

2. Complete both histograms in `kernel.cu`: the kernels `basic_histo_kernel()` and `privatized_histo_kernel()`, and the host functions `basicHistogram()` and `privatizedHistogram()` that launch them. There should be no changes necessary for `main.cu`. `bins` is already set to zero before your kernel runs. The input size and number of bins can be any size, but we will not test your code with more than 4096 bins or an input larger than 1M elements. **Please use a block size of 512** (`BLOCK_SIZE` in `kernel.cu`) for both kernels.

3. There are three modes of operation for the application. Check `main()` for a description of the modes (repeated below). Both versions support each of these modes.

   - **No arguments**: The application will create a randomly initialized vector of 1 million elements and 4096 bins. After the device histogram is invoked, it will compute the correct solution using the CPU, and compare that solution with the device-computed solution. If every bin matches, it will print out "TEST PASSED" to the screen before exiting.
   - **One argument**: The application will create a randomly initialized vector of the specified size. The number of bins will be 4096.
   - **Two arguments**: The first argument will specify the input size, and the second argument specifies the number of bins.

   To ensure consistency when grading, do not change the `srand` seed value.

4. Build and test on Bender. `make` builds both versions: `./histogram-basic` runs the basic kernel and `./histogram-privatized` runs the privatized kernel.

```
make
./histogram-basic                    # basic, 1,000,000 elements, 4096 bins
./histogram-privatized               # privatized, 1,000,000 elements, 4096 bins
./histogram-privatized 1000000 1024  # privatized, 1,000,000 elements, 1024 bins
./histogram-basic 1025 513           # basic, 1025 elements, 513 bins
```

Each program checks every bin against the CPU and prints `TEST PASSED`, or lists the first wrong bins, the total count, and a hint. It also prints `Kernel time`, the kernel's run time measured on the GPU. Bender's CUDA tools may not be on your PATH; if `nvcc` is not found, add `export PATH=/usr/local/cuda/bin:$PATH` to `~/.bashrc`. To check for problems that can go unnoticed otherwise, run your programs under `compute-sanitizer`:

```
compute-sanitizer ./histogram-basic 1025 513                    # out-of-bounds accesses
compute-sanitizer ./histogram-privatized 1025 513               # out-of-bounds accesses
```

```
compute-sanitizer --tool racecheck ./histogram-privatized 10000 4096   # non-atomic updates of shared memory
compute-sanitizer --tool synccheck ./histogram-privatized 10000 33     # some __syncthreads() in a divergent b
```

5. Commit and push your completed histogram code to the private repository.

Dynamic shared memory

The size of the private histogram is the number of bins, which is a command line argument. Therefore, it is only known when the program runs and not when it is compiled. A shared memory array declared with a fixed size, such as `__shared__ unsigned int hist_s[512];`, needs its size at compile time. Since this assignment can have variable bin size, we need to use **dynamic shared memory**, whose size you choose when you launch the kernel:

1. In the kernel, declare the array with `extern` and no size. Its size comes from the launch. All `extern __shared__` arrays of a kernel start at the same address, so use a single one for the histogram:

```
extern __shared__ unsigned int hist_s[];
```

2. When you launch the kernel, pass the number of bytes of shared memory each block needs as the third launch parameter, after the grid and block sizes:

```
my_kernel<<<dim_grid, dim_block, num_bins * sizeof(unsigned int)>>>(...);
```

Inside the kernel, `hist_s` then has room for exactly `num_bins` counters: don't index it past `num_bins - 1`. For more details and examples, see Using Shared Memory in CUDA C/C++: Dynamic Shared Memory.

Since we won't test more than 4096 bins, a fixed-size array of 4096 counters (16 KB) would also work, but every block would then reserve 16 KB of shared memory even for a histogram of a few bins.

GPGPU-Sim analysis of atomic operations

> Please see the GPGPU-Sim setup guide.

We will now count the global memory atomic operations of the basic and privatized kernels using GPGPU-Sim. Remember to `make clean && make` inside the GPGPU-Sim image first, and again on Bender before running on the real GPU. Inside the image, `make` builds for the simulated GTX 480 (`sm_20`), because the default architecture of the image's CUDA 4.0 compiler has no `atomicAdd`. Copy the configuration files into your repository once (`cp -a ~/gpgpu-sim_distribution/configs/GTX480/* .`), and save the output of each run to a file, e.g. `./histogram-basic 20000 256 &> basic-256.out`. Do NOT commit any output files to the git repository.

Simulating is slow, so use an input of 20,000 elements. For kernel timing, use `gpu_sim_cycle`, the number of cycles the kernel took.

Global atomic operations vs global atomic requests

GPGPU-Sim does not report global atomic *operations* directly; it reports global atomic *requests*. The two are different:

- A **global atomic operation** is one thread's `atomicAdd` on a location in global memory. (You can compute the expected global atomic operation by hand.)

- A **global atomic request** is one packet that an SM sends to the memory partitions (the L2 cache and DRAM). One request can carry the atomic operations of several threads. (For example, multiple operations can be coalesced into fewer requests.)

When a warp executes an `atomicAdd` to global memory, the simulator groups its threads' operations into requests, like it coalesces ordinary loads and stores. Each half of the warp (16 threads) is handled separately. So one request carries 1 to 16 operations.

The simulator reports the **requests** sent from the SMs to the memory partitions as:

```
traffic_breakdown_coretomem[GLOBAL_ATOMIC] = 45608 {40:8,72:629,}
```

The first number is the total bytes. The braces list `packet size:count` pairs: here 8 requests of 40 bytes and 629 requests of 72 bytes, so 637 global atomic requests in total (8 x 40 + 629 x 72 = 45,608 bytes).
`traffic_breakdown_memtocore[GLOBAL_ATOMIC]` counts the replies that come back, one per request.

Note: atomic operations on shared memory never leave the SM, so they don't appear to global memory at all; they are counted in `gpgpu_n_shmem_insn` with the other shared memory instructions.

Report

Answer the questions in `REPORT.md` in your repository. Expand "How to use this file" at the top of it for how to add plots, tables and math. Save plots as PNG files in `plots/` and embed them in `REPORT.md`. Do NOT commit GPGPU-Sim output files.

Submission

1. Fill in your name and answers in `REPORT.md`, and add any plots to `plots/`.

2. Commit and push your completed basic and privatized histogram code (`kernel.cu`), `REPORT.md` and `plots/` to your GitHub repository. Open `REPORT.md` on GitHub to check that it displays correctly; that's the version we grade.
3. Check that the automated tests pass on your final commit.

© 2019 - 2026 Daniel Wong